

Détection de proximité

Nous avons vu comment calculer la distance entre deux objets en utilisant la fonction `Vector2.Distance()` et en lançant un rayon avec `Physics2D.Raycast()`. Une autre méthode couramment utilisée pour détecter la proximité entre des objets est l'utilisation de cercles de détection avec la fonction `Physics2D.OverlapCircle()` et `Physics2D.OverlapCircleAll()`.

Cela peut être particulièrement utile pour des situations où vous souhaitez détecter si un objet se trouve à l'intérieur d'une certaine zone circulaire autour d'un point donné, comme pour détecter un ou des ennemis proches, des objets ramassables, ou des zones d'effet, lancer un sort qui affecte une zone au complet, etc.

La différence principale entre `OverlapCircle` et `Raycast` est que le premier vérifie une zone entière (le cercle) tandis que le second vérifie une ligne droite (le rayon). Cela permet de détecter plusieurs objets en même temps et de gérer des zones d'influence plus larges.

Il existe plusieurs variantes :

- `Physics2D.OverlapBox()`, `Physics2D.OverlapBoxAll()`, etc. pour des zones rectangulaires.
- `Physics.OverlapSphere()`, `Physics.OverlapSphereAll()` pour la 3D.

[Documentation officielle de `OverlapCircle`](#)

1.1 Utilisation de `OverlapCircle`

La fonction `Physics2D.OverlapCircle()` permet de vérifier si **UN** collider se trouve à l'intérieur d'un cercle défini par un centre et un rayon.

```
void DetecterProximite(Vector2 positionCentre, float rayon)
{
    Collider2D colliderProche = Physics2D.OverlapCircle(positionCentre, rayon);
    if (colliderProche != null)
    {
        Debug.Log("Un objet est proche : " + colliderProche.name);
    }
    else
    {
        Debug.Log("Aucun objet proche.");
    }
}
```